

ASSIGNMENT 4

Submitted By: Aiswarya AC

Submitted To: Subin P John

1. Differentiate between StringBuffer and StringBuilder in Java. Mention at least two key differences.

StringBuffer

- Thread-Safe: Methods are synchronized
- Slower because of synchronization

StringBuilder

- Not Thread-Safe
- Faster performance

2. Write a Java program to demonstrate Type Casting (implicit and explicit) using variables of type int, double, and char.

Program:

```
package Assignment4;
```

```
public class TypeCasting {
```

```
    public static void main(String[] args) {
```

```
        //    IMPLICIT CASTING (small type to large type)
```

```
        int num = 100;
```

```
        double d = num;
```

```
        System.out.println("Integer value: " + num);
```

```
        System.out.println("Converted to double: " + d);
```

```
        //    EXPLICIT CASTING (large type to small type)
```

```
        double value = 99.99;
```

```
        int i = (int) value;
```

```
        System.out.println("\nDouble value: " + value);
```

```
        System.out.println("Converted to int: " + i);

//    CHAR TO INT

    char ch = 'A';

    int ascii = ch;

    System.out.println("\nCharacter: " + ch);

    System.out.println("ASCII value: " + ascii);

    }

}
```

Output:

Integer value: 100

Converted to double: 100.0

Double value: 99.99

Converted to int: 99

Character: A

ASCII value: 65

3. What are Wrapper Classes in Java? Write a short Java program to convert a primitive int into an Integer object and then back to a primitive.

Wrapper Classes are used to convert primitive data types into objects.

Example:

- int —————> Integer
- char —————> Character
- double ———> Double

Uses:

- Required when using collections like ArrayList
- Help in converting between strings and primitives
- Allow use of null values for primitive types

Program:

```
package Assignment4;

public class WrapperClass {

    public static void main(String[] args) {

        // PRIMITIVE int

        int num = 50;

        // BOXING (Primitive to Object)

        Integer obj = Integer.valueOf(num);

        //UNBOXING (object to primitive)

        int value = obj.intValue();
```

```
        System.out.println("Primitive int: " + num);

        System.out.println("Integer object: " + obj);

        System.out.println("Converted back to primitive: " + value);

    }

}
```

Output:

Primitive int: 50

Integer object: 50

Converted back to primitive: 50

4. Write a Java program to demonstrate the difference between a static method and a non-static method.

Program:

```
package Assignment4;

public class StaticNonStatic {

    // STATIC METHOD

    static void staticMethod() {

        System.out.println("This is a static method");

    }

    //NON STATIC METHOD

    void nonStaticMethod() {

        System.out.println("This is a non-static method");

    }

    public static void main(String[] args) {

        // Calling static method directly

        staticMethod();

        // Creating object for non-static method

        StaticNonStatic obj = new StaticNonStatic();

        obj.nonStaticMethod();

    }

}
```

Output:

This is a static method

This is a non-static method

- Static methods belong to the class
- Non-static methods belong to objects
- Static methods can be called without creating an object

5. Explain Inheritance in OOP. Write a Java program to demonstrate Single Level Inheritance.

Inheritance is an OOP concept where one class acquires the properties and methods of another class.

Parent class —→ Superclass

Child class —→ Subclass

Single-level inheritance means one child class inherits from one parent class.

Program:

```
package Assignment4;
```

```
// PARENT CLASS
```

```
class Animal {  
    void sound () {  
        System.out.println("Animal makes a sound");  
    }  
}
```

```
// CHILD CLASS
```

```
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}
```

```
public class Inheritance {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
    }  
}
```

```
//INHERITED METHOD
```

```
d.sound();
```

```
//CHILD CLASS METHOD
```

```
d.bark();
```

```
}
```

```
}
```

Output:

Animal makes a sound

Dog barks